

Jetpack Compose第五章第一节教学讲义

(Compose与View协同及导航)

各位同学，前面我们已经掌握了Compose的UI构建、动画和手势交互能力，但实际开发中很少有项目是“纯Compose”或“纯传统View”的——老项目需要用Compose逐步重构新功能，新项目也可能需要集成传统View的成熟组件（如地图、播放器）。此外，任何APP都离不开页面跳转和导航管理。这节课咱们就聚焦“Compose与View的协同工作”和“Compose导航体系”，解决实际开发中的“混合开发”和“页面跳转”核心问题。

一、核心认知：Compose与View的关系

Compose是Google推出的现代UI工具包，但它并非要彻底取代传统View（如TextView、Button），而是提供了“和平共处”的能力。两者的关系可以总结为：**相互兼容、无缝集成、按需选用**。

实际开发中的常见场景：

- 老项目（View体系）：在Activity/Fragment中嵌入Compose组件开发新功能，无需重构整个项目；
- 新项目（Compose体系）：在Compose界面中嵌入传统View组件，复用成熟功能（如第三方地图SDK）；
- 主题统一：让Compose组件和View组件使用相同的主题样式，保证APP视觉一致性；
- 页面导航：用Compose的导航组件管理所有页面（包括View页面和Compose页面）的跳转。

二、基础集成：View体系中嵌入Compose

对于用传统View开发的老项目，想要引入Compose开发新功能，核心是通过 `ComposeView` 在Activity/Fragment中嵌入Compose内容。这是最常见的“增量升级”场景。

1. 前置准备：配置依赖

首先需要在项目的 `build.gradle`（Module级别）中添加Compose相关依赖，确保View能识别Compose组件：

Code block

```
1 // 1. Compose基础依赖
2 implementation "androidx.compose.ui:ui:1.5.3"
3 implementation "androidx.compose.material3:material3:1.2.0"
4 implementation "androidx.compose.ui:ui-tooling-preview:1.5.3"
```

```
5 // 2. View中嵌入Compose的核心依赖
6 implementation "androidx.compose.ui:ui-viewbinding:1.5.3"
7 // 3. 预览工具依赖（仅开发环境）
8 debugImplementation "androidx.compose.ui:ui-tooling:1.5.3"
9 debugImplementation "androidx.compose.ui:ui-test-manifest:1.5.3"
10 // 4. Kotlin编译器插件
11 kotlinCompilerPluginClasspath "org.jetbrains.kotlin:kotlin-gradle-plugin:1.9.0"
12
```



注意：依赖版本需与项目的Kotlin版本匹配，可在Android Studio的Project Structure中统一管理版本号，避免版本冲突。

2. 实战1：在Activity中嵌入Compose组件

步骤：① 在布局文件中添加 `ComposeView`；② 在Activity中通过 `setContent` 设置Compose内容。

第一步：创建包含ComposeView的布局文件（res/layout/activity_main.xml）

Code block

```
1 <LinearLayout
2     xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:orientation="vertical"
6     android:padding="16dp">
7
8
9     <TextView
10         android:id="@+id/tv_title"
11         android:layout_width="match_parent"
12         android:layout_height="wrap_content"
13         android:text="这是传统View的标题"
14         android:textSize="20sp"
15         android:textStyle="bold"/>
16
17
18     <androidx.compose.ui.platform.ComposeView
19         android:id="@+id/compose_container"
20         android:layout_width="match_parent"
21         android:layout_height="0dp">
```

```

22         android:layout_weight="1"
23         android:layout_marginTop="16dp"/>
24
25     </LinearLayout>

```

第二步：在Activity中初始化ComposeView并设置内容

Code block

```

1  import android.os.Bundle
2  import androidx.appcompat.app.AppCompatActivity
3  import androidx.compose.foundation.layout.Column
4  import androidx.compose.foundation.layout.padding
5  import androidx.compose.material3.Button
6  import androidx.compose.material3.Text
7  import androidx.compose.ui.Modifier
8  import androidx.compose.ui.unit.dp
9  import androidx.compose.ui.platform.ComposeView
10
11  class MainActivity : AppCompatActivity() {
12      override fun onCreate(savedInstanceState: Bundle?) {
13          super.onCreate(savedInstanceState)
14          setContentView(R.layout.activity_main)
15
16          // 1. 找到布局中的ComposeView
17          val composeView = findViewById<ComposeView>(R.id.compose_container)
18
19          // 2. 给ComposeView设置Compose内容（和纯Compose项目的setContent一致）
20          composeView.setContent {
21              // 这里是Compose的UI代码
22              Column(modifier = Modifier.padding(16.dp)) {
23                  Text(text = "这是Compose的文本组件", modifier =
24                      Modifier.padding(bottom = 8.dp))
25                  Button(onClick = {
26                      // 点击事件：修改传统View的内容
27                      findViewById<TextView>(R.id.tv_title).text = "Compose触发了
28                      View更新"
29                  }) {
30                      Text(text = "点击修改传统View标题")
31                  }
32              }
33      }

```

核心知识点

- `ComposeView`：是连接View体系和Compose体系的“桥梁”，本质是一个传统View，可像TextView一样放在布局文件中；
- `setContent`：ComposeView的核心方法，用于传入Compose的UI内容，内部逻辑和纯Compose项目的Activity一致；
- 双向通信：Compose组件可以通过 `findViewById` 操作传统View（如修改TextView文本），传统View也可以通过回调触发Compose状态更新。

3. 实战2：在Fragment中嵌入Compose组件

Fragment中嵌入Compose的逻辑与Activity类似，核心是在 `onViewCreated` 中初始化ComposeView：

Code block

```
1  import android.os.Bundle
2  import android.view.LayoutInflater
3  import android.view.View
4  import android.view.ViewGroup
5  import androidx.compose.material3.Text
6  import androidx.compose.ui.platform.ComposeView
7  import androidx.fragment.app.Fragment
8
9  class ComposeFragment : Fragment() {
10     override fun onCreateView(
11         inflater: LayoutInflater,
12         container: ViewGroup?,
13         savedInstanceState: Bundle?
14     ): View? {
15         // 方式1: 直接返回ComposeView (无需布局文件)
16         return ComposeView(requireContext()).apply {
17             setContent {
18                 Text(text = "Fragment中直接嵌入的Compose内容")
19             }
20         }
21
22         // 方式2: 从布局文件加载 (类似Activity, 需在布局中添加ComposeView)
23         // val rootView = inflater.inflate(R.layout.fragment_compose,
24         //                                 container, false)
25         // val composeView = rootView.findViewById<ComposeView>
26         // (R.id.fragment_compose_view)
27         // composeView.setContent { ... }
28         // return rootView
29     }
30 }
```

三、反向集成：Compose中使用传统View

在纯Compose项目中，当需要复用传统View组件（如 `WebView`、`MapView` 或第三方View组件）时，核心是通过 `AndroidView` 组件将传统View“包裹”成Compose可识别的组件。

1. 实战3：在Compose中嵌入TextView和WebView

步骤：① 通过 `AndroidView` 的 `factory` 创建传统View；② 通过 `update` 方法更新View的状态（如文本、链接）。

Code block

```

1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.foundation.layout.fillMaxSize
3  import androidx.compose.foundation.layout.padding
4  import androidx.compose.material3.Button
5  import androidx.compose.material3.Text
6  import androidx.compose.runtime.getValue
7  import androidx.compose.runtime.mutableStateOf
8  import androidx.compose.runtime.remember
9  import androidx.compose.runtime.setValue
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.platform.LocalContext
12 import androidx.compose.ui.viewinterop.AndroidView
13 import androidx.compose.ui.unit.dp
14 import android.webkit.WebView
15 import android.widget.TextView
16
17 @Composable
18 fun ComposeWithViewDemo() {
19     // 状态：控制传统View的内容（Compose状态驱动View更新）
20     var textContent by remember { mutableStateOf("初始文本") }
21     var webUrl by remember { mutableStateOf("https://www.google.com") }
22
23     Column(modifier = Modifier.fillMaxSize().padding(16.dp)) {
24         // 1. Compose组件：按钮（触发状态变化）
25         Button(
26             onClick = {
27                 textContent = "Compose状态更新了View"
28                 webUrl = "https://developer.android.com/jetpack/compose"
29             },
30             modifier = Modifier.padding(bottom = 16.dp)
31         ) {
32             Text(text = "更新传统View内容")

```

```

33     }
34
35     // 2. 嵌入传统TextView (通过AndroidView)
36     AndroidView(
37         // factory: 创建传统View的工厂方法
38         factory = { context ->
39             TextView(context).apply {
40                 textSize = 18f
41                 setTextColor(android.graphics.Color.BLACK)
42             }
43         },
44         // update: 当Compose状态变化时, 更新View的内容
45         update = { textView ->
46             textView.text = textContent
47         },
48         modifier = Modifier.padding(bottom = 16.dp)
49     )
50
51     // 3. 嵌入传统WebView (加载网页)
52     AndroidView(
53         factory = { context ->
54             WebView(context).apply {
55                 settings.javaScriptEnabled = true // 启用JS (按需设置)
56             }
57         },
58         update = { webView ->
59             if (webView.url != webUrl) {
60                 webView.loadUrl(webUrl)
61             }
62         },
63         modifier = Modifier.weight(1f)
64     )
65 }
66 }

```

核心知识点

- AndroidView**：Compose提供的“View包装器”，用于将传统View转换为Compose组件，接收两个核心参数：
 - factory**：创建传统View的逻辑，只在组件初始化时执行一次；
- update**：当依赖的Compose状态变化时执行，用于更新View的属性（如文本、链接），类似Compose的重组逻辑。

LocalContext：Compose提供的获取当前上下文的方法，替代传统View中的 **this** 或 **requireContext()**；

状态驱动：传统View的更新完全由Compose状态控制，无需手动调用 `setText`、`loadUrl` 等方法，符合Compose的状态驱动思想。

四、视觉统一：Compose与View共用主题

为了保证APP视觉风格一致，Compose组件和传统View必须使用相同的主题（如颜色、字体、圆角）。Compose支持两种主题共用方式：① 从View主题（`Theme.MaterialComponents`）映射到Compose主题；② 统一使用Compose主题。

1. 实战4：从View主题映射到Compose主题

如果项目已使用传统View主题（如Material Design 2），可通过 `MaterialTheme` 的 `colors`、`typography` 参数映射主题属性：

第一步：传统View主题（res/values/themes.xml）

Code block

```
1  <resources>
2      <style name="AppTheme"
3          parent="Theme.MaterialComponents.DayNight.NoActionBar">
4          <item name="colorPrimary">#6200EE</item>
5
6          <item name="colorAccent">#03DAC6</item>
7
8          <item name="android:textColorPrimary">#000000</item>
9
10         <item
11             name="android:textAppearanceLarge">@style/AppTextAppearance.Large</item>
12         </style>
13         <style name="AppTextAppearance.Large"
14             parent="TextAppearance.MaterialComponents.Large">
15             <item name="android:textSize">24sp</item>
16             <item name="android:textStyle">bold</item>
17         </style>
18     </resources>
```

第二步：Compose主题映射View主题属性

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.material3.MaterialTheme
```

```

3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.Composable
5  import androidx.compose.ui.Modifier
6  import androidx.compose.ui.graphics.Color
7  import androidx.compose.ui.platform.LocalContext
8  import androidx.compose.ui.text.TextStyle
9  import androidx.compose.ui.unit.sp
10 import android.content.Context
11 import android.util.TypedValue
12
13 // 1. 从View主题中获取颜色 (如colorPrimary)
14 @Composable
15 fun getColorFromViewTheme(context: Context, attrResId: Int): Color {
16     val typedValue = TypedValue()
17     context.theme.resolveAttribute(attrResId, typedValue, true)
18     return Color(typedValue.data)
19 }
20
21 // 2. 从View主题中获取字体样式 (如textAppearanceLarge)
22 @Composable
23 fun getTextStyleFromViewTheme(context: Context, attrResId: Int): TextStyle {
24     val typedValue = TypedValue()
25     context.theme.resolveAttribute(attrResId, typedValue, true)
26     val textAppearance = TypedValue.complexToResourceId(typedValue.data)
27     // 这里简化处理, 实际可通过TextAppearance解析更详细属性
28     return TextStyle(
29         fontSize = 24.sp,
30         fontWeight = androidx.compose.ui.text.font.FontWeight.Bold,
31         color = getColorFromViewTheme(context, android.R.attr.textColorPrimary)
32     )
33 }
34
35 // 3. 共用主题的Compose组件
36 @Composable
37 fun ThemedComposeDemo() {
38     val context = LocalContext.current
39     // 从View主题映射颜色和字体
40     val primaryColor = getColorFromViewTheme(context,
41 com.google.android.material.R.attr.colorPrimary)
42     val accentColor = getColorFromViewTheme(context,
43 com.google.android.material.R.attr.colorAccent)
44     val largeTextStyle = getTextStyleFromViewTheme(context,
45 android.R.attr.textAppearanceLarge)
46
47     // Compose主题: 使用View主题的属性
48     MaterialTheme(
49         colors = androidx.compose.material3.MaterialTheme.colorScheme.copy(

```

```

47         primary = primaryColor,
48         secondary = accentColor
49     ),
50     typography = androidx.compose.material3.MaterialTheme.typography.copy(
51         headlineLarge = largeTextStyle
52     )
53 ) {
54     Column(modifier = Modifier.padding(16.dp)) {
55         Text(
56             text = "Compose使用View主题的标题",
57             style = MaterialTheme.typography.headlineLarge,
58             color = MaterialTheme.colorScheme.primary
59         )
60         Text(
61             text = "Compose使用View主题的强调色",
62             modifier = Modifier.padding(top = 8.dp),
63             color = MaterialTheme.colorScheme.secondary
64         )
65     }
66 }
67 }

```

2. 推荐方案：统一使用Compose主题

如果是新项目，建议直接使用Compose的Material 3主题，然后通过 `Context` 将Compose主题属性传递给传统View：

Code block

```

1  import androidx.compose.material3.MaterialTheme
2  import androidx.compose.material3.LightColorScheme
3  import androidx.compose.runtime.Composable
4  import androidx.compose.ui.graphics.Color
5  import android.widget.TextView
6
7  // 1. 定义Compose全局主题
8  private val LightColorScheme = lightColorScheme(
9      primary = Color(0xFF6200EE),
10     secondary = Color(0xFF03DAC6),
11     surface = Color(0xFFFFFBE),
12     onPrimary = Color.White,
13     onSecondary = Color.Black,
14     onSurface = Color(0xFF1C1B1F),
15 )
16
17 @Composable

```

```

18 fun AppComposeTheme(
19     content: @Composable () -> Unit
20 ) {
21     val colorScheme = LightColorScheme
22     MaterialTheme(
23         colorScheme = colorScheme,
24         typography = MaterialTheme.typography,
25         content = content
26     )
27 }
28
29 // 2. 传统View使用Compose主题颜色
30 @Composable
31 fun TextViewWithComposeTheme(textView: TextView) {
32     val colorScheme = MaterialTheme.colorScheme
33
34     textView.setTextColor(android.graphics.Color.parseColor(colorScheme.primary.toString()))
35     textView.textSize = MaterialTheme.typography.headlineLarge.fontSize.value
36 }
37
38 // 3. 用法：在Compose中包裹所有内容
39 @Composable
40 fun UnifiedThemeDemo() {
41     AppComposeTheme {
42         // 内部的Compose组件和传统View都会使用统一主题
43         Column {
44             Text(text = "Compose主题文本", color =
45                 MaterialTheme.colorScheme.primary)
46             AndroidView(
47                 factory = { TextView(it) },
48                 update = { TextViewWithComposeTheme(it) }
49             )
50         }
51     }
52 }

```

五、页面导航：Compose Navigation组件

无论是纯Compose项目还是混合项目，页面跳转都需要统一的导航管理。Compose官方提供的 `Navigation` 组件，支持：① Compose页面之间的跳转；② Compose与View页面（Activity/Fragment）的跳转；③ 传递参数；④ 回退栈管理。

1. 前置准备：添加Navigation依赖

```
1 // Compose Navigation核心依赖
2 implementation "androidx.navigation:navigation-compose:2.7.0"
3 // 如果需要跳转Activity, 需添加传统Navigation依赖
4 implementation "androidx.navigation:navigation-fragment-ktx:2.7.0"
5 implementation "androidx.navigation:navigation-ui-ktx:2.7.0"
```

2. 核心概念

- `NavController`：导航控制器，管理所有页面的跳转、回退和参数传递，是导航的核心；
- `NavGraph`：导航图，定义所有页面（目的地）和页面之间的跳转关系；
- `NavDestination`：导航目的地，即单个页面（Compose组件或Activity/Fragment）；
- `action`：页面之间的跳转动作，定义从A页面到B页面的路径。

3. 实战5：Compose页面之间的导航

实现两个Compose页面（HomeScreen和DetailScreen）的跳转，并传递参数（如商品ID）。

第一步：创建两个Compose页面

Code block

```
1 import androidx.compose.foundation.layout.Column
2 import androidx.compose.foundation.layout.padding
3 import androidx.compose.material3.Button
4 import androidx.compose.material3.Text
5 import androidx.compose.runtime.Composable
6 import androidx.compose.ui.Modifier
7 import androidx.compose.ui.unit.dp
8 import androidx.navigation.NavController
9
10 // 1. 首页 (传递参数到详情页)
11 @Composable
12 fun HomeScreen(navController: NavController) {
13     Column(modifier = Modifier.padding(16.dp)) {
14         Text(text = "首页", modifier = Modifier.padding(bottom = 16.dp))
15         // 跳转详情页, 并传递参数 (商品ID=123)
16         Button(onClick = {
17             navController.navigate("detail/123") {
18                 // 可选: 添加跳转动画
19                 launchSingleTop = true // 避免重复创建相同页面
20             }
21         }) {
22             Text(text = "跳转到详情页 (商品ID=123)")
23         }
24     }
25 }
```

```

25 }
26
27 // 2. 详情页 (接收首页传递的参数)
28 @Composable
29 fun DetailScreen(navController: NavController, productId: String) {
30     Column(modifier = Modifier.padding(16.dp)) {
31         Text(text = "详情页", modifier = Modifier.padding(bottom = 16.dp))
32         Text(text = "接收的商品ID: $productId", modifier =
33             Modifier.padding(bottom = 16.dp))
34         // 回退到首页
35         Button(onClick = { navController.popBackStack() }) {
36             Text(text = "返回首页")
37         }
38     }
39 }

```

第二步：创建导航图并初始化NavController

Code block

```

1  import androidx.compose.foundation.layout.fillMaxSize
2  import androidx.compose.material3.MaterialTheme
3  import androidx.compose.material3.Surface
4  import androidx.compose.runtime.Composable
5  import androidx.compose.ui.Modifier
6  import androidx.navigation.compose.NavHost
7  import androidx.navigation.compose.composable
8  import androidx.navigation.compose.rememberNavController
9
10 @Composable
11 fun AppNavigation() {
12     // 1. 创建导航控制器 (rememberNavController确保状态不丢失)
13     val navController = rememberNavController()
14
15     // 2. 导航图：定义页面关系
16     NavHost(
17         navController = navController,
18         startDestination = "home" // 初始页面 (首页)
19     ) {
20         // 首页目的地
21         composable(route = "home") {
22             HomeScreen(navController = navController)
23         }
24         // 详情页目的地：通过{productId}定义参数占位符
25         composable(route = "detail/{productId}") { backStackEntry ->
26             // 从回退栈中获取参数

```

```

27         val productId = backStackEntry.arguments?.getString("productId")
28         ?: "未知ID"
29         DetailScreen(navController = navController, productId = productId)
30     }
31 }
32
33 // 3. 在主Activity中使用导航
34 class NavigationActivity : AppCompatActivity() {
35     override fun onCreate(savedInstanceState: Bundle?) {
36         super.onCreate(savedInstanceState)
37         setContent {
38             AppComposeTheme {
39                 Surface(
40                     modifier = Modifier.fillMaxSize(),
41                     color = MaterialTheme.colorScheme.surface
42                 ) {
43                     AppNavigation() // 启动导航
44                 }
45             }
46         }
47     }
48 }

```

4. 实战6：Compose跳转至传统Activity

在Compose页面中通过 `NavController` 跳转至传统View的Activity，并传递参数：

Code block

```

1  import androidx.compose.foundation.layout.padding
2  import androidx.compose.material3.Button
3  import androidx.compose.material3.Text
4  import androidx.compose.runtime.Composable
5  import androidx.compose.ui.Modifier
6  import androidx.compose.ui.platform.LocalContext
7  import androidx.compose.ui.unit.dp
8  import androidx.navigation.compose.NavHost
9  import androidx.navigation.compose.composable
10 import androidx.navigation.compose.rememberNavController
11 import androidx.navigation.navArgument
12 import android.content.Intent
13
14 // 1. 传统View的Activity (接收参数)
15 class ViewActivity : AppCompatActivity() {
16     override fun onCreate(savedInstanceState: Bundle?) {

```

```

17         super.onCreate(savedInstanceState)
18         setContentView(R.layout.activity_view)
19         // 接收Compose传递的参数
20         val message = intent.getStringExtra("message") ?: "无参数"
21         findViewById<TextView>(R.id.tv_view_message).text = message
22     }
23 }
24
25 // 2. Compose页面中添加跳转逻辑
26 @Composable
27 fun ComposeToViewScreen(navController: NavController) {
28     val context = LocalContext.current
29     Button(
30         onClick = {
31             // 方式1: 直接通过Intent跳转 (简单场景)
32             val intent = Intent(context, ViewActivity::class.java)
33             intent.putExtra("message", "来自Compose的参数")
34             context.startActivity(intent)
35
36             // 方式2: 通过Navigation组件跳转 (需配置导航图, 适合复杂导航)
37             // navController.navigate("viewActivity")
38         },
39         modifier = Modifier.padding(16.dp)
40     ) {
41         Text(text = "Compose跳转至传统Activity")
42     }
43 }
44
45 // 3. (可选) 在导航图中配置Activity目的地
46 @Composable
47 fun MixedNavigation() {
48     val navController = rememberNavController()
49     NavHost(navController = navController, startDestination = "composeToView")
50     {
51         composable(route = "composeToView") {
52             ComposeToViewScreen(navController = navController)
53         }
54         // 配置Activity目的地
55         activity(route = "viewActivity") {
56             // 传递参数
57             navArgument("message") { type = NavType.StringType }
58             // 关联Activity
59             destination = ViewActivity::class.java
60         }
61     }

```

六、高级能力：深层链接（Deep Link）

深层链接是指通过URL直接打开APP的指定页面（如从短信链接直接打开商品详情页）。Compose的Navigation组件完美支持深层链接，无论是Compose页面还是View页面都可以配置。

1. 实战7：为Compose页面配置深层链接

第一步：在AndroidManifest.xml中配置深层链接

Code block

```
1  <activity
2      android:name=".NavigationActivity"
3      android:exported="true">
4      <intent-filter>
5          <action android:name="android.intent.action.MAIN" />
6          <category android:name="android.intent.category.LAUNCHER" />
7      </intent-filter>
8
9      <intent-filter>
10         <action android:name="android.intent.action.VIEW" />
11         <category android:name="android.intent.category.DEFAULT" />
12         <category android:name="android.intent.category.BROWSABLE" />
13
14         <data
15             android:scheme="app"
16             android:host="myapp"
17             android:pathPattern="/detail/.*" />
18     </intent-filter>
19 </activity>
```

第二步：在导航图中关联深层链接

Code block

```
1  import androidx.navigation.compose.composable
2  import androidx.navigation.navDeepLink
3
4  @Composable
5  fun DeepLinkNavigation() {
6      val navController = rememberNavController()
7      NavHost(navController = navController, startDestination = "home") {
8          composable(
9              route = "detail/{productId}",
10             // 关联深层链接
11             deepLinks = listOf(
```

```
12         navDeepLink {
13             // 匹配的URL模板
14             uriPattern = "app://myapp/detail/{productId}"
15         }
16     )
17     ) { backStackEntry ->
18         val productId = backStackEntry.arguments?.getString("productId")
19         ?: "未知"
20         DetailScreen(navController = navController, productId = productId)
21     }
22 }
```

测试方式

1. 在Android Studio的Terminal中执行命令：

```
adb shell am start -a android.intent.action.VIEW -d
"app://myapp/detail/456"
```

2. 执行后APP会直接打开详情页，显示商品ID=456。

七、小节回顾：核心知识点与API速查表

功能场景	核心API/组件	关键步骤
View中嵌入Compose	ComposeView、setContent	1. 布局中添加ComposeView；2. Activity中setContent设置Compose内容
Compose中嵌入View	AndroidView、factory、update	1. factory创建传统View；2. update监听状态更新View属性
主题统一	MaterialTheme、getColorFromViewTheme	1. 从View主题映射属性；2. 统一使用Compose主题
Compose页面导航	NavController、NavHost、composable	1. 创建NavController；2. 配置NavGraph；3. 用navigate跳转
Compose跳转View页面	Intent、activity(route)	1. 用Intent直接跳转；2. 或在导航图中配置Activity目的地
深层链接	navDeepLink、intent-filter	

	1. Manifest配置intent-filter; 2. 导航图关联deepLink
--	--

开发避坑指南

1. 依赖版本统一：Compose、Navigation、Kotlin的版本必须匹配，否则会出现编译错误，建议用 `ext` 或 `dependencyResolutionManagement` 统一管理版本；
2. NavController创建时机：必须在 `Composable` 函数中通过 `rememberNavController` 创建，确保与Compose的生命周期同步；
3. 参数传递安全：从回退栈获取参数时，必须判空（如 `?: "默认值"`），避免参数缺失导致崩溃；
4. 传统View内存管理：在Compose中嵌入 `WebView` 等耗内存组件时，需在 `onDispose` 中释放资源（如 `webView.destroy()`）；
5. 深层链接测试：必须配置 `BROWSABLE` 类别，否则无法通过URL唤醒APP。

八、课后作业：动手实现“混合导航APP”

需求：基于本节课知识点，实现一个包含“Compose页面+传统View页面”的混合导航APP，功能如下：

1. 基础架构：
使用Compose的Navigation组件管理所有页面导航；
2. 统一主题：Compose组件和传统View使用相同的主色调（#6200EE）和标题字体（24sp加粗）；
3. 页面组成：
首页（Compose）：显示“混合导航演示”标题，包含两个按钮：“跳转到Compose详情页”“跳转到传统View页面”；
4. Compose详情页（Compose）：接收首页传递的“用户名”参数（默认“Guest”），显示欢迎文本，提供“返回首页”和“跳转到WebView页面”按钮；
5. 传统View页面（Activity+XML）：显示“这是传统View页面”文本，接收Compose传递的“消息”参数，提供“返回上一页”按钮；
6. WebView页面（Compose中嵌入WebView）：加载“<https://developer.android.com>”，提供“返回详情页”按钮；
7. 进阶功能：
为Compose详情页配置深层链接，URL格式为“app://mixnav/detail?username=张三”，点击该URL可直接打开详情页并显示“张三”；
8. 传统View页面的背景色和文本颜色与Compose主题保持一致；

9. 技术要求：

用ComposeView在传统Activity中嵌入WebView；

10. 用AndroidView在Compose中嵌入传统View（可选）；

11. 用Navigation组件实现所有页面跳转，包括参数传递；

12. 主题统一，视觉风格一致；

13. 处理参数缺失的异常情况，避免崩溃。

提示：深层链接测试可通过adb命令或创建HTML页面添加链接进行。重点练习“混合页面导航”和“主题统一”，确保Compose和View组件无缝协同，导航逻辑清晰易懂。

（注：文档部分内容可能由 AI 生成）